Stoage Management / Disk Scheduling

Overview

A Process makes the I/O requests to the operating system to access the disk. Disk Scheduling Algorithm manages those requests and decides the order of the disk access given to the requests.

Disk scheduling is a technique used by the operating system to schedule multiple requests for **accessing the disk**.

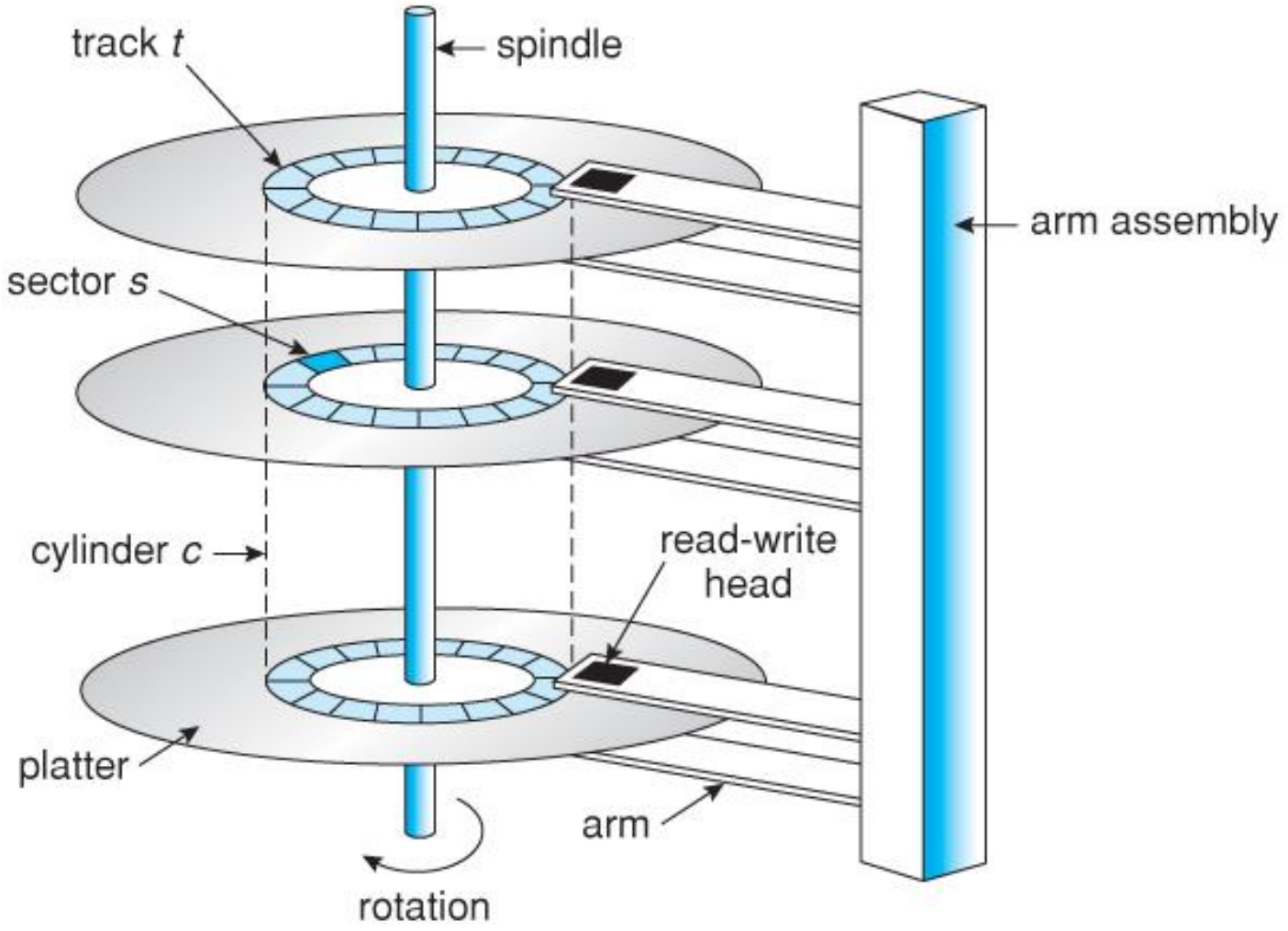
Why Disk Scheduling is needed?

- **Disk Scheduling Algorithms are needed because a process can make multiple I/O requests and multiple processes run at the same time.**
- **The requests made by a process may be located at different sectors on different tracks. Due to this, the seek time may increase more.**
- **These algorithms help in minimizing the seek time by ordering the requests made by the processes.**

Important Terms

- **Seek Time -** It is the time taken by the disk arm to locate the desired track.
- **Rotational Latency -** The time taken by a desired sector of the disk to rotate itself to the position where it can access the Read/Write heads is called Rotational Latency.
- **Transfer Time -** It is the time taken to transfer the data requested by the processes (disk to CPU).
- **Disk Access Time -** Disk Access time is the sum of the Seek Time, Rotational Latency, and Transfer Time.

Magnetic Disk



Magnetic Disk

• One or more ***platters*** in the form of disks covered with magnetic media.

• Each platter has two working ***surfaces.***

• Each working surface is divided into a number of concentric rings called ***tracks.*** The collection of all tracks that are the same distance from the edge of the platter, is called a ***cylinder***.

• Each track is further divided into ***sectors,*** traditionally containing 512 bytes of data each, although some modern disks occasionally use larger sector sizes. (header, trailer, bad sector)

• The data on a hard drive is read by read-write ***heads.*** The standard configuration uses one head per surface, each on a separate ***arm***, and controlled by a common ***arm assembly*** which moves all heads simultaneously from one cylinder to another. (technical difficulties)

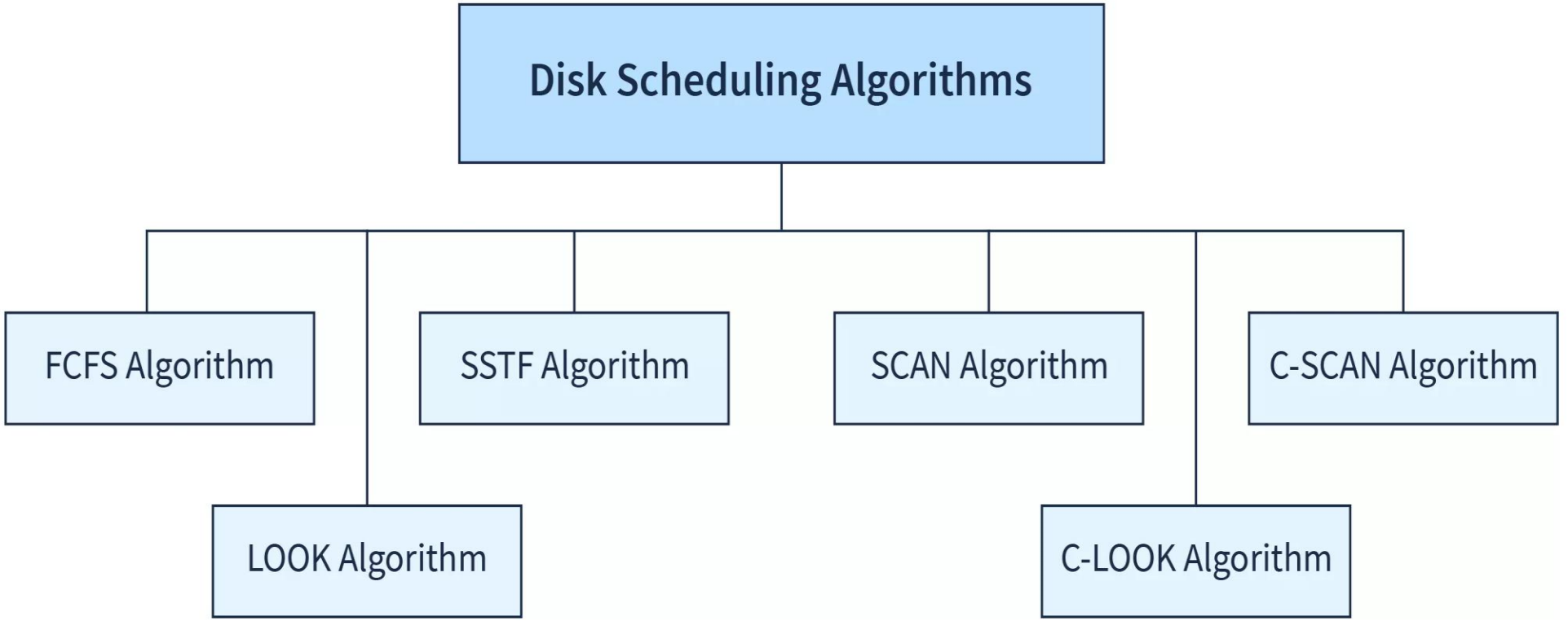
• The storage capacity of a traditional disk drive is equal to the number of heads (i.e. the number of working surfaces), times the number of tracks per surface, times the number of sectors per track, times the number of bytes per sector.

• A particular physical block of data is specified by providing the head-sector-cylinder number at which it is located.

Disk Scheduling

- Disk transfer speeds are limited primarily by ***seek times*** and ***rotational latency.*** When multiple requests are to be processed there is also some inherent delay in waiting for other requests to be processed.
- ***Bandwidth*** is measured by the amount of data transferred divided by the total amount of time from the first request being made to the last transfer being completed, (for a series of disk requests.)
- Both bandwidth and access time can be improved by processing requests in a good order.
- Disk requests include the disk address, memory address, number of sectors to transfer, and whether the request is for reading or writing.

Types of Disk Scheduling Algorithm

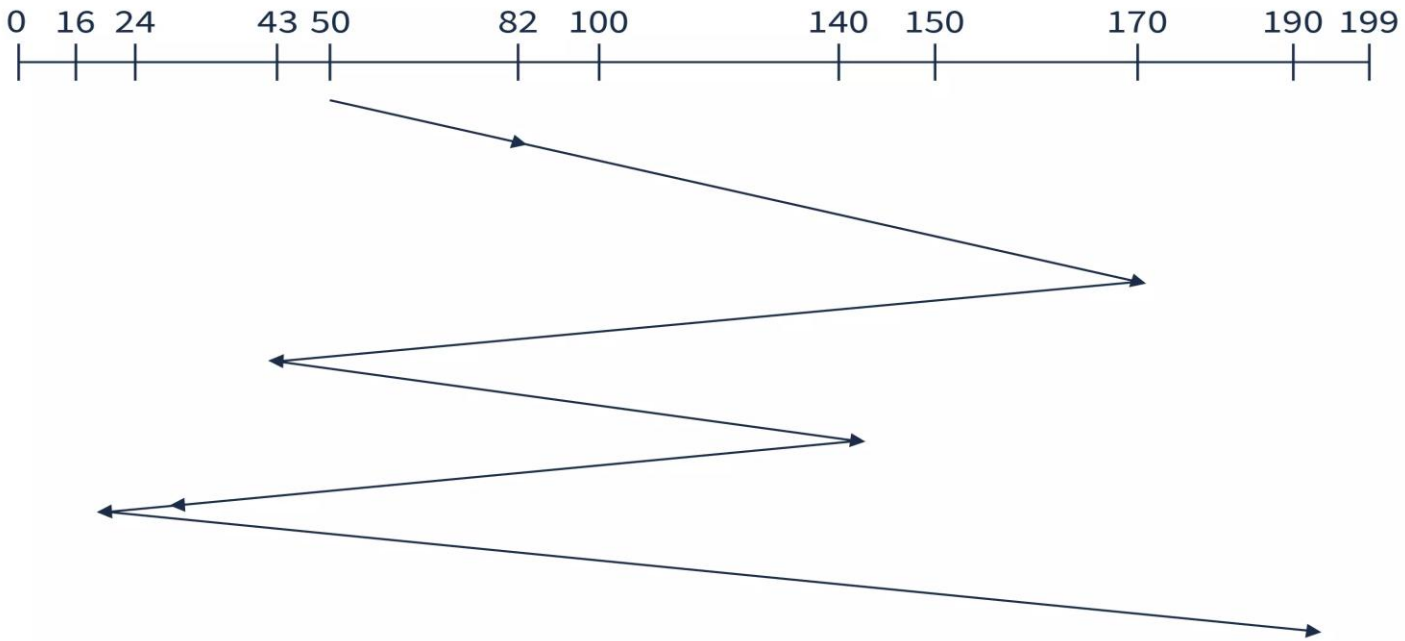


First Come First Serve Algorithm

It stands for 'first-come-first-serve'. As the name suggests, the request which comes first will be processed first and so on. The requests coming to the disk are arranged in a proper sequence as they arrive. Since every request is processed in this algorithm, so there is no chance of 'starvation'.

First Come First Serve Algorithm

Example: Suppose a disk having 200 tracks (0-199). The request sequence(82,170,43,140,24,16,190) of disk are shown as in the given figure and the head start is at request 50.



Seek time= (82-50) + (170-82) + (170-43) + (140-43) + (140-24) + (24-16) + (190-16) = 642

First Come First Serve Algorithm

Advantages:

- Implementation is easy.
- No chance of starvation.

Disadvantages:

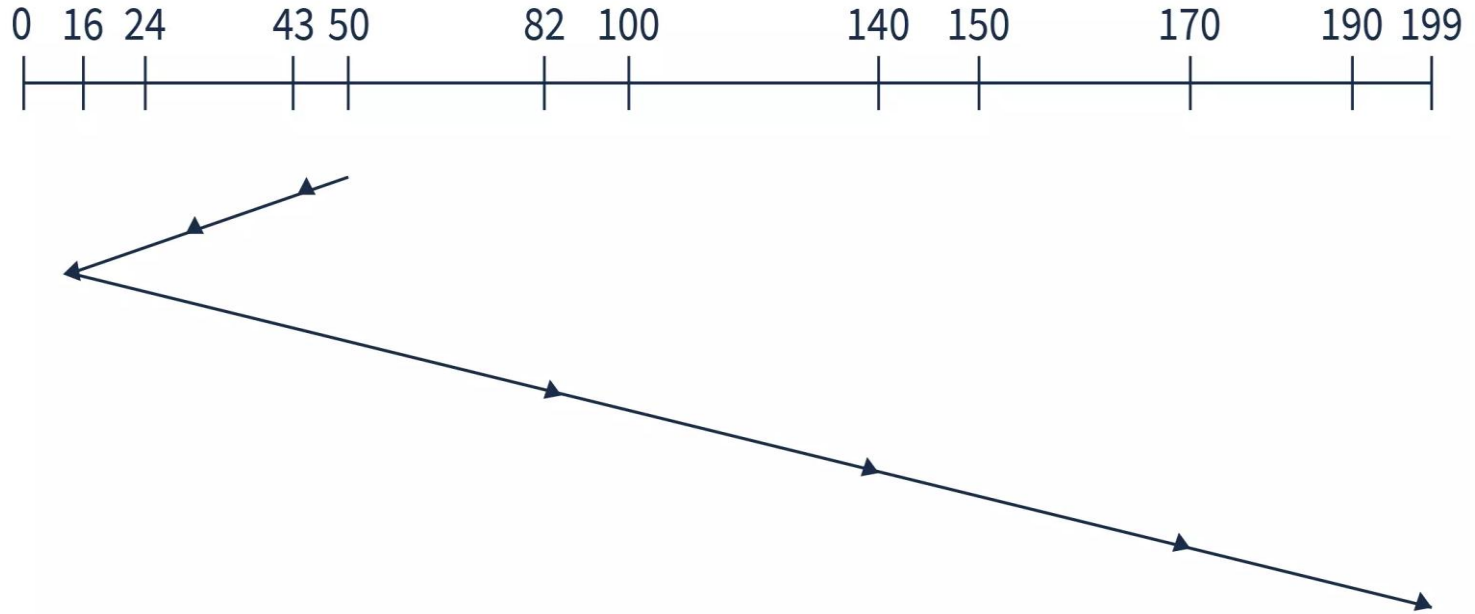
- 'Seek time' higher.
- Not so efficient.

Shortest Seek Time First

It stands for 'Shortest seek time first'. As the name suggests, it searches for the request having the least 'seek time' and executes them first. This algorithm has less 'seek time' as compared to FCFS Algorithm.

Shortest Seek Time First

Example: Suppose a disk having 200 tracks (0-199). The request sequence(82,170,43,140,24,16,190) of disk are shown as in the given figure and the head start is at request 50.



Seek time= (50-43) + (43-24) + (24-16) + (82-16) + (140-82) + (170-140) + (190-170) = 208

Shortest Seek Time First

Advantages:

- In this algorithm, disk response time is less.
- More efficient than FCFS.

Disadvantages:

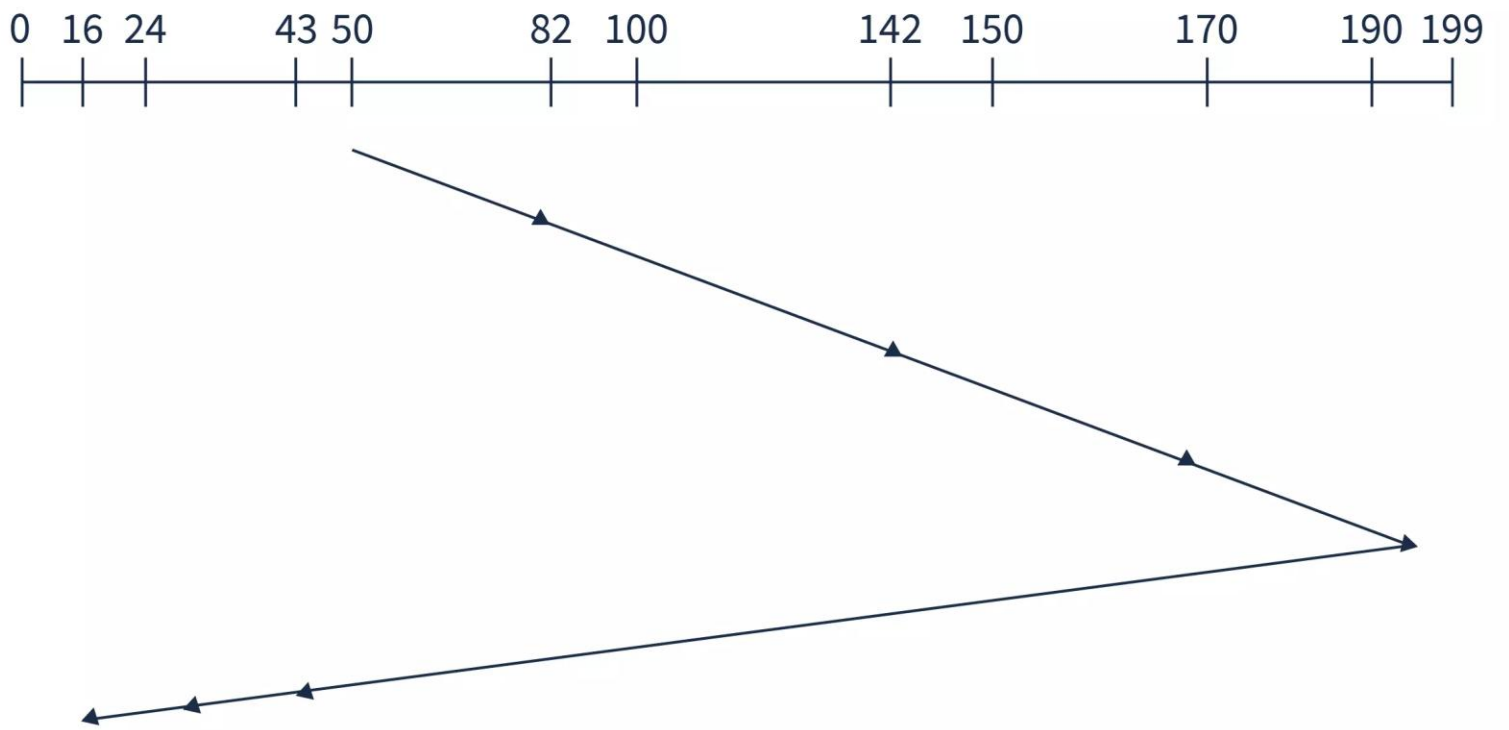
- Less speed of algorithm execution.
- Starvation can be seen.

SCAN Disk Scheduling Algorithm

In this algorithm, the head starts to scan all the requests in a direction and reaches the end of the disk. After that, it reverses its direction and starts to scan again the requests in its path and serves them. Due to this feature, this algorithm is also known as the "Elevator Algorithm".

SCAN Disk Scheduling Algorithm

Example: Suppose a disk having 200 tracks (0-199). The request sequence(82,170,43,140,24,16,190) of disk are shown as in the given figure and the head start is at request 50.



Seek time= (199-50) + (199-16) =332

SCAN Disk Scheduling Algorithm

Advantages:

- Implementation is easy.
- Requests do not have to wait in a queue.

Disadvantage:

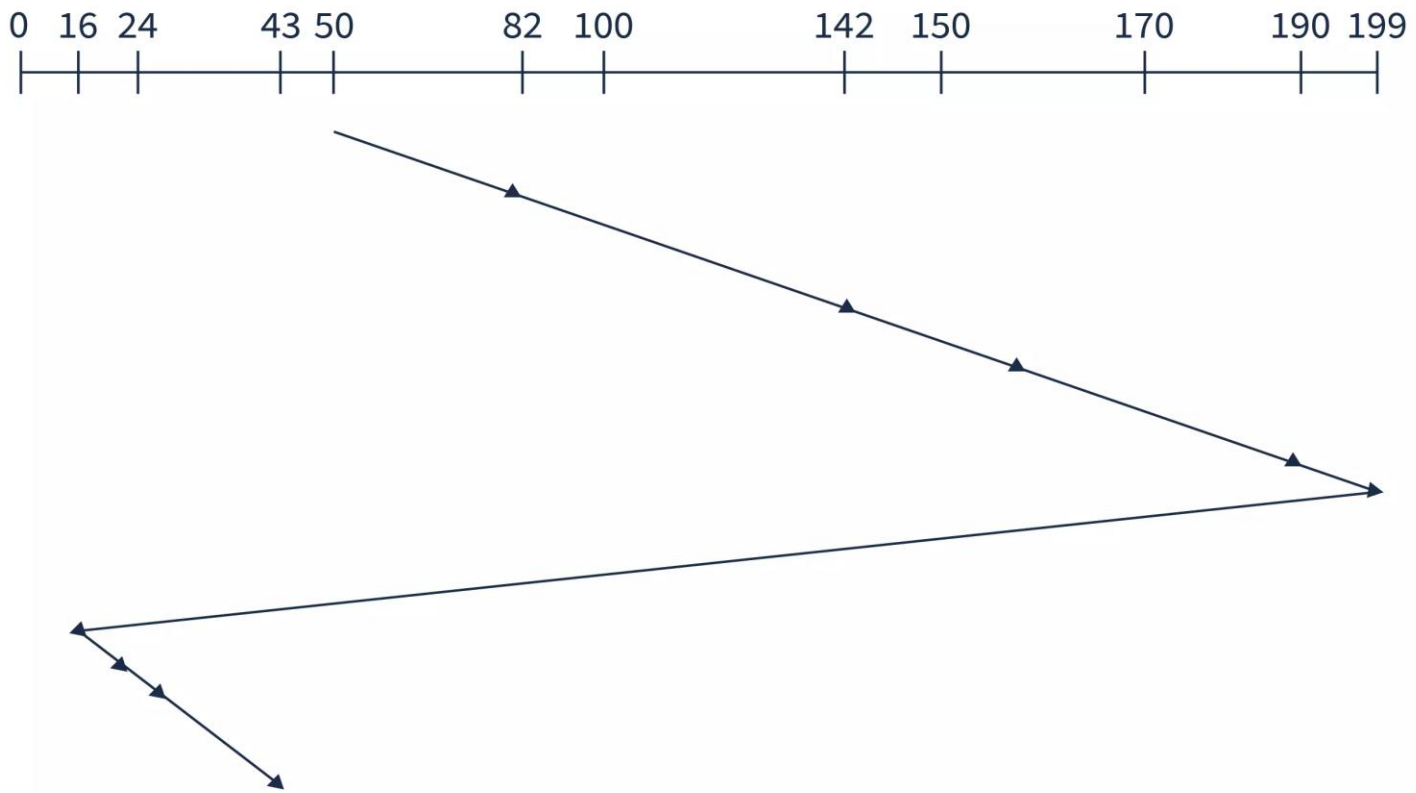
- The head keeps going on to the end even if there are no requests in that direction.

C-SCAN Disk Scheduling Algorithm

- It stands for "Circular-Scan".
- This algorithm is almost the same as the Scan disk algorithm but one thing that makes it different is that 'after reaching the one end and reversing the head direction, it starts to come back.
- The disk arm moves toward the end of the disk and serves the requests coming into its path.
- After reaching the end of the disk it reverses its direction and again starts to move to the other end of the disk but while going back it does not serve any requests.

C-SCAN Disk Scheduling Algorithm

Example: Suppose a disk having 200 tracks (0-199). The request sequence(82,170,43,140,24,16,190) of disk are shown as in the given figure and the head start is at request 50.



Seek time= (199-50) + (199-0) + (43-0) =391

C-SCAN Disk Scheduling Algorithm

Advantages:

- The waiting time is uniformly distributed among the requests.
- Response time is good in it.

Disadvantages:

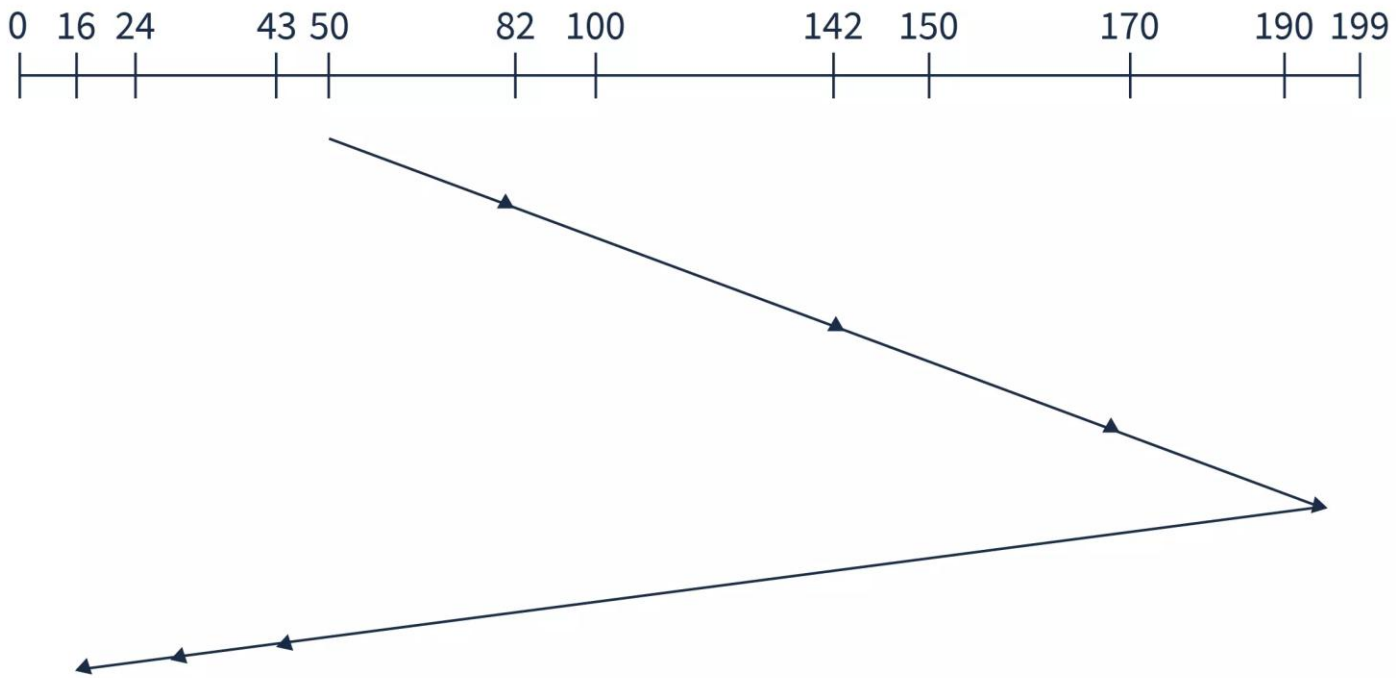
- The time taken by the disk arm to locate a spot is increased here.
- The head keeps going to the end of the disk.

LOOK Disk Scheduling Algorithm

In this algorithm, the disk arm moves to the 'last request' present and services them. After reaching the last requests, it reverses its direction and again comes back to the starting point. It does not go to the end of the disk, in spite, it goes to the end of requests.

LOOK Disk Scheduling Algorithm

Example: Suppose a disk having 200 tracks (0-199). The request sequence(82,170,43,140,24,16,190) of disk are shown as in the given figure and the head start is at request 50.



Seek time= (190-50) + (190-16) = 314

LOOK Disk Scheduling Algorithm

Advantages:

- Starvation does not occur.
- Since the head does not go to the end of the disk, the time is not wasted here.

Disadvantage:

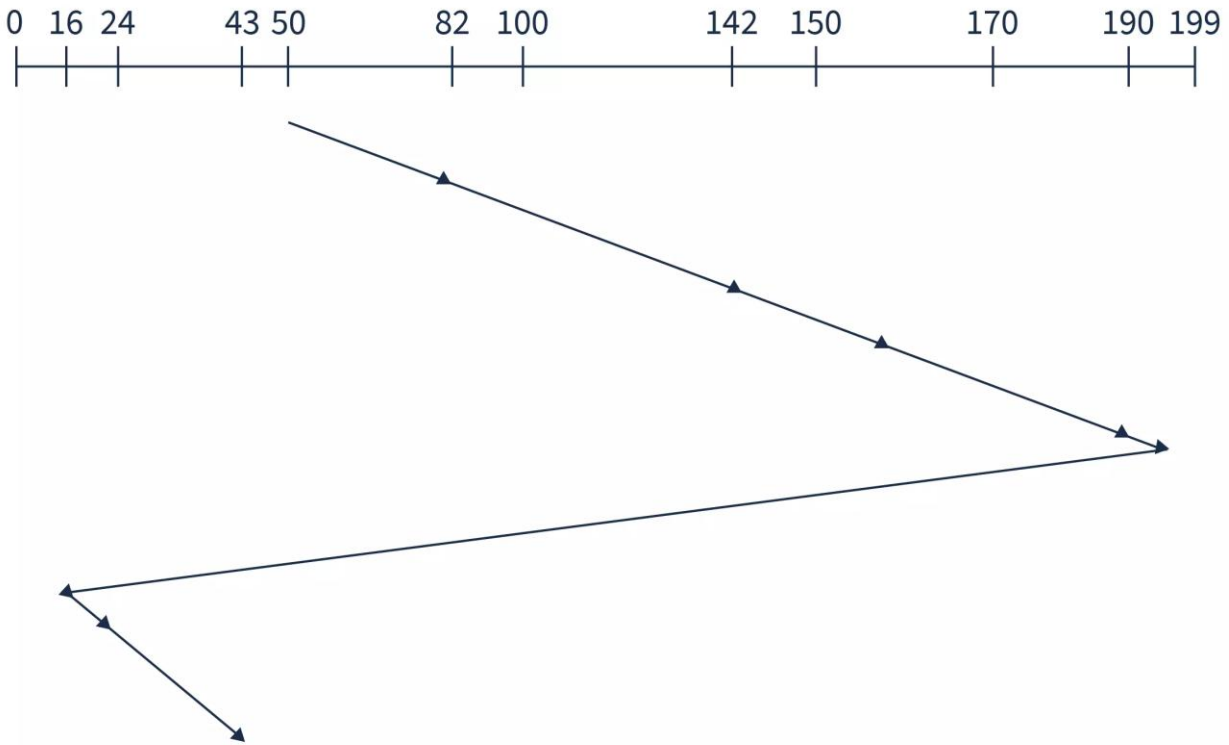
- The arm has to be conscious to find the last request

C-LOOK Disk Scheduling Algorithm

The C-Look algorithm is almost the same as the Look algorithm. The only difference is that after reaching the end requests, it reverses the direction of the head and starts moving to the initial position. But in moving back, it does not serve any requests.

C-LOOK Disk Scheduling Algorithm

Example: Suppose a disk having 200 tracks (0-199). The request sequence(82,170,43,140,24,16,190) of disk are shown as in the given figure and the head start is at request 50.



Seek time= (190-50) + (190-16) + (43-16) =341

C-LOOK Disk Scheduling Algorithm

Advantages:

- The waiting time is decreased.
- If there are no requests till the end, it reverses the head direction immediately.
- Starvation does not occur.
- The time taken by the disk arm to find the desired spot is less.

Disadvantage:

- The arm has to be conscious about finding the last request.

Overview

A Process makes the I/O requests to the operating system to access the disk. **Disk Scheduling Algorithm** manages those requests and decides the order of the disk access given to the requests.

Disk scheduling is a technique used by the operating system to schedule multiple requests for **accessing the disk**.

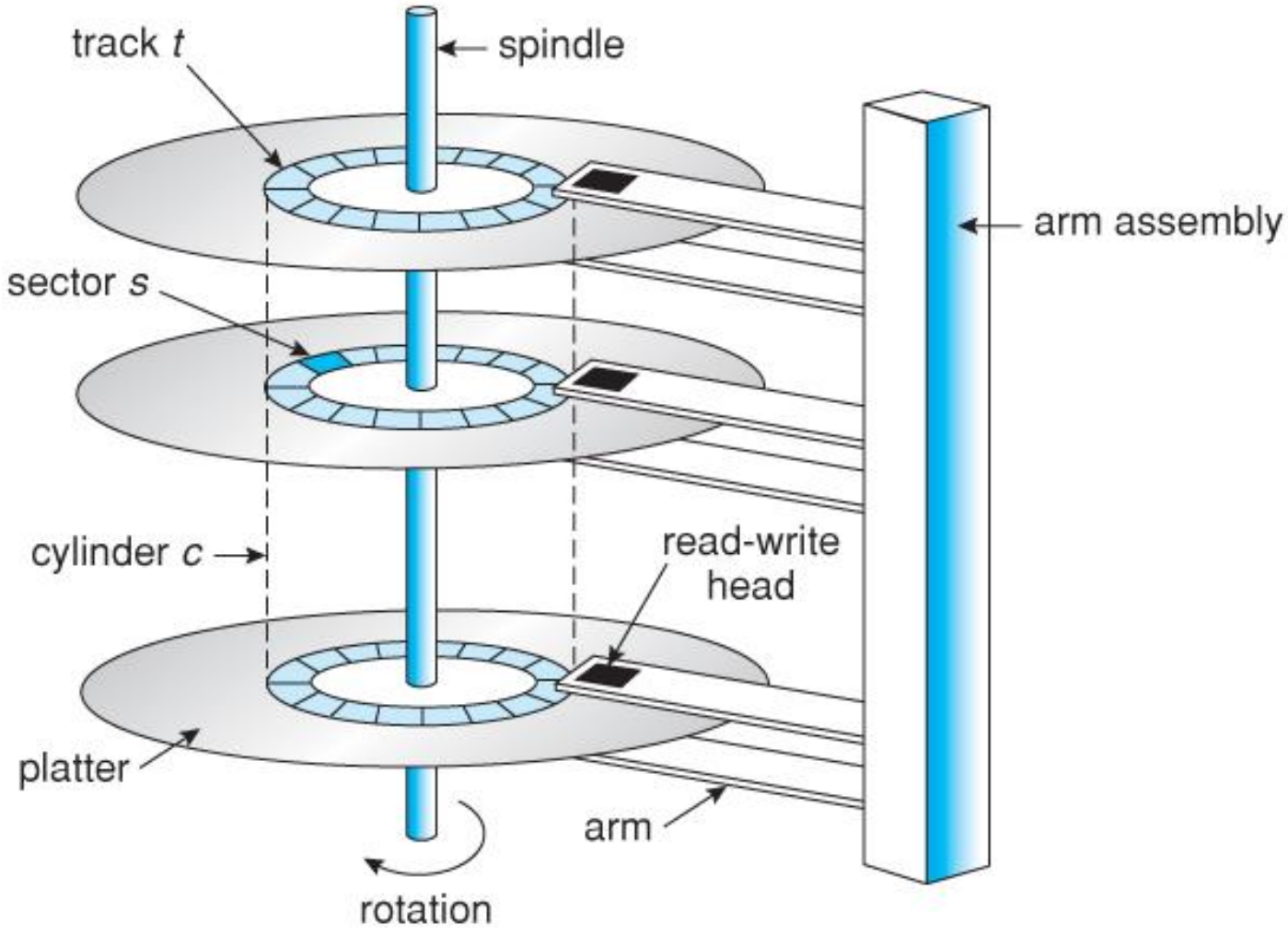
Why Disk Scheduling is needed?

- Disk Scheduling Algorithms are needed because a process can make multiple I/O requests and multiple processes run at the same time.
- The requests made by a process may be located at different sectors on different tracks. Due to this, the seek time may increase more.
- These algorithms help in minimizing the seek time by ordering the requests made by the processes.

Important Terms

- **Seek Time -** It is the time taken by the disk arm to locate the desired track.
- **Rotational Latency -** The time taken by a desired sector of the disk to rotate itself to the position where it can access the Read/Write heads is called Rotational Latency.
- **Transfer Time -** It is the time taken to transfer the data requested by the processes (disk to CPU).
- **Disk Access Time -** Disk Access time is the sum of the Seek Time, Rotational Latency, and Transfer Time.

Magnetic Disk



Magnetic Disk

• One or more ***platters*** in the form of disks covered with magnetic media.

• Each platter has two working ***surfaces.***

• Each working surface is divided into a number of concentric rings called ***tracks.*** The collection of all tracks that are the same distance from the edge of the platter, is called a ***cylinder***.

• Each track is further divided into ***sectors,*** traditionally containing 512 bytes of data each, although some modern disks occasionally use larger sector sizes. (header, trailer, bad sector)

• The data on a hard drive is read by read-write ***heads.*** The standard configuration uses one head per surface, each on a separate ***arm***, and controlled by a common ***arm assembly*** which moves all heads simultaneously from one cylinder to another. (technical difficulties)

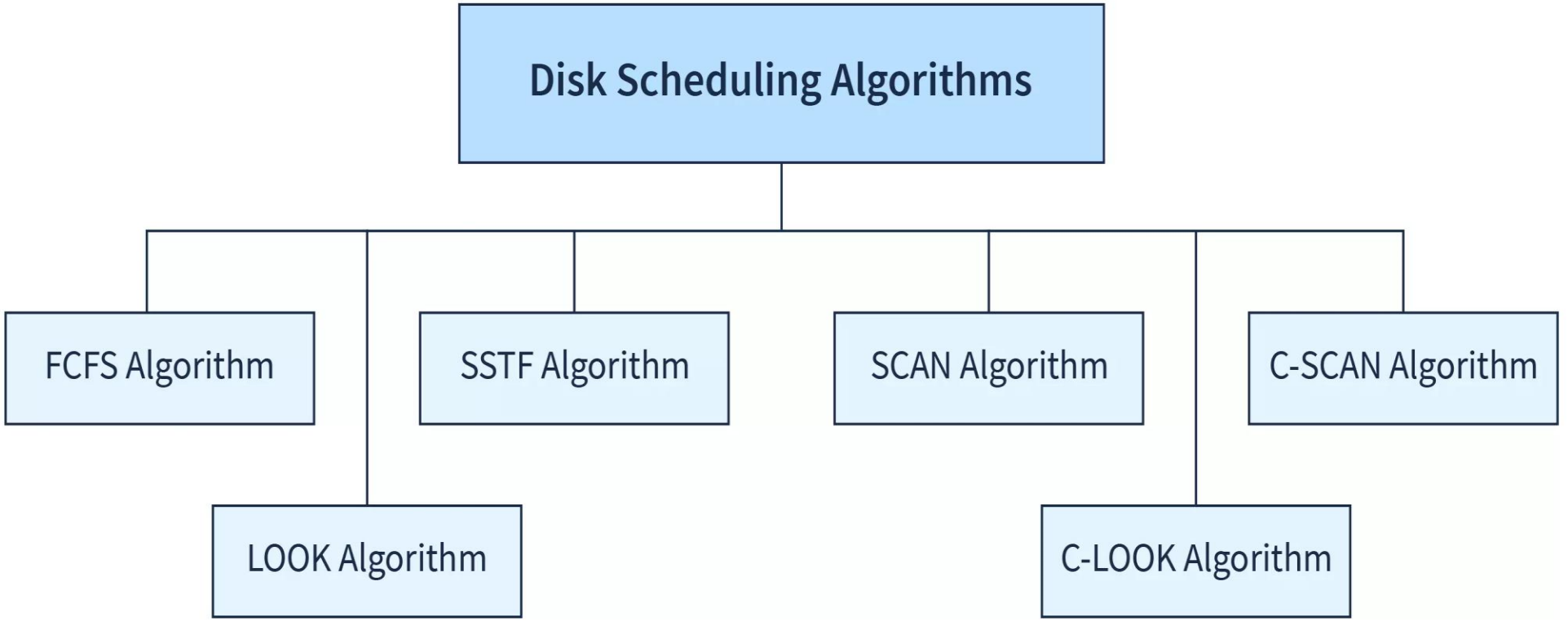
• The storage capacity of a traditional disk drive is equal to the number of heads (i.e. the number of working surfaces), times the number of tracks per surface, times the number of sectors per track, times the number of bytes per sector.

• A particular physical block of data is specified by providing the head-sector-cylinder number at which it is located.

Disk Scheduling

- Disk transfer speeds are limited primarily by ***seek times*** and ***rotational latency.*** When multiple requests are to be processed there is also some inherent delay in waiting for other requests to be processed.
- ***Bandwidth*** is measured by the amount of data transferred divided by the total amount of time from the first request being made to the last transfer being completed, (for a series of disk requests.)
- Both bandwidth and access time can be improved by processing requests in a good order.
- Disk requests include the disk address, memory address, number of sectors to transfer, and whether the request is for reading or writing.

Types of Disk Scheduling Algorithm

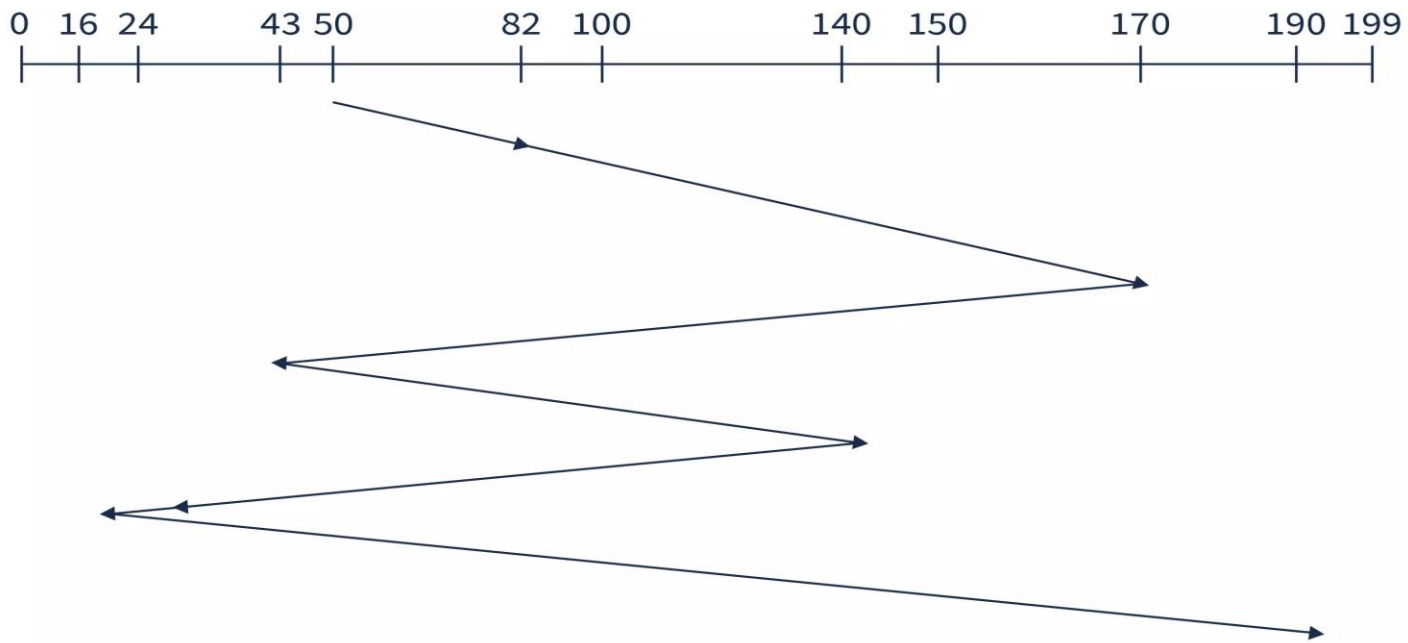


First Come First Serve Algorithm

It stands for 'first-come-first-serve'. As the name suggests, the request which comes first will be processed first and so on. The requests coming to the disk are arranged in a proper sequence as they arrive. Since every request is processed in this algorithm, so there is no chance of 'starvation'.

First Come First Serve Algorithm

Example: Suppose a disk having 200 tracks (0-199). The request sequence(82,170,43,140,24,16,190) of disk are shown as in the given figure and the head start is at request 50.



Seek time= (82-50) + (170-82) + (170-43) + (140-43) + (140-24) + (24-16) + (190-16) = 642

First Come First Serve Algorithm

Advantages:

- Implementation is easy.
- No chance of starvation.

Disadvantages:

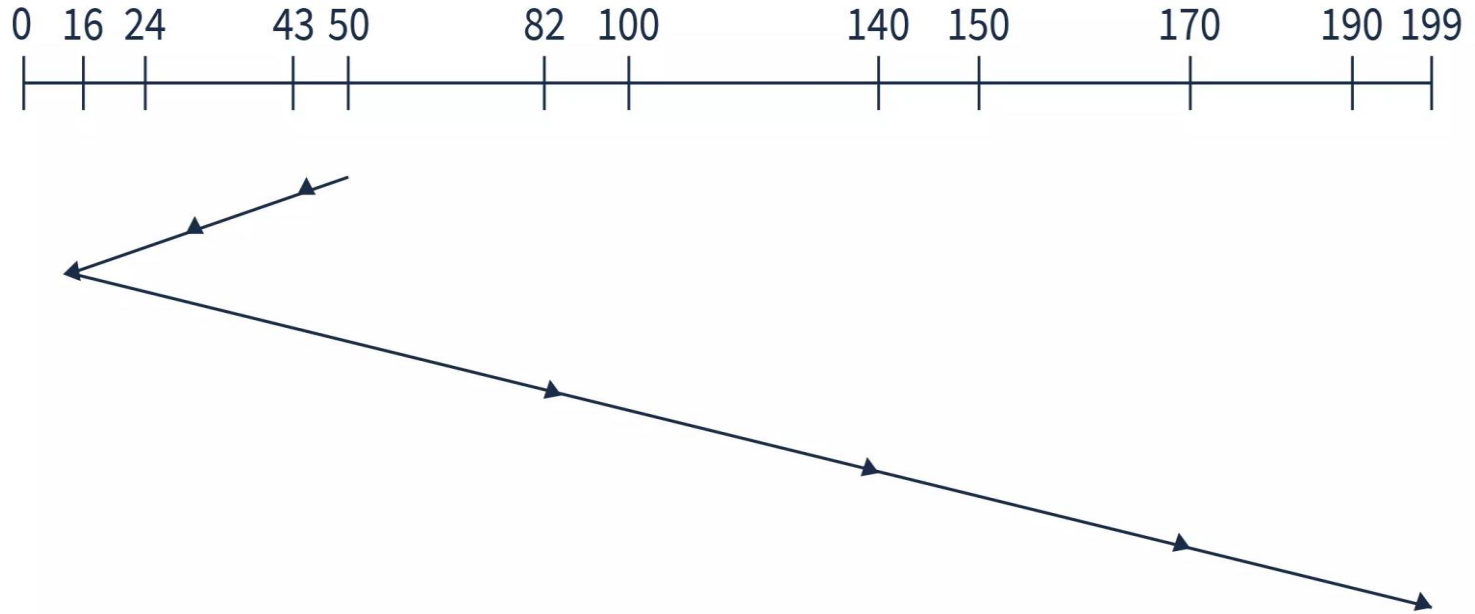
- 'Seek time' higher.
- Not so efficient.

Shortest Seek Time First

It stands for 'Shortest seek time first'. As the name suggests, it searches for the request having the least 'seek time' and executes them first. This algorithm has less 'seek time' as compared to FCFS Algorithm.

Shortest Seek Time First

Example: Suppose a disk having 200 tracks (0-199). The request sequence(82,170,43,140,24,16,190) of disk are shown as in the given figure and the head start is at request 50.



Seek time= (50-43) + (43-24) + (24-16) + (82-16) + (140-82) + (170-140) + (190-170) = 208

Shortest Seek Time First

Advantages:

- In this algorithm, disk response time is less.
- More efficient than FCFS.

Disadvantages:

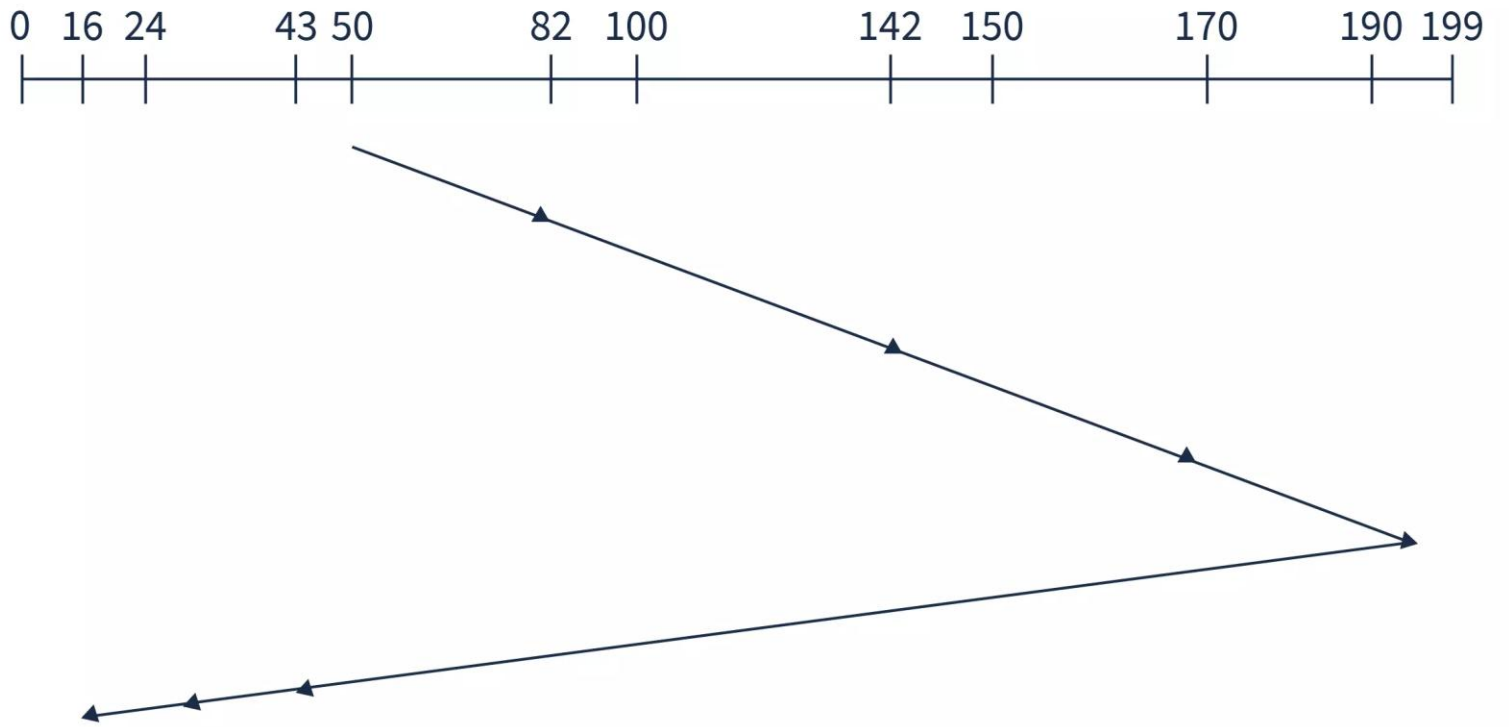
- Less speed of algorithm execution.
- Starvation can be seen.

SCAN Disk Scheduling Algorithm

In this algorithm, the head starts to scan all the requests in a direction and reaches the end of the disk. After that, it reverses its direction and starts to scan again the requests in its path and serves them. Due to this feature, this algorithm is also known as the "Elevator Algorithm".

SCAN Disk Scheduling Algorithm

Example: Suppose a disk having 200 tracks (0-199). The request sequence(82,170,43,140,24,16,190) of disk are shown as in the given figure and the head start is at request 50.



Seek time= (199-50) + (199-16) =332

SCAN Disk Scheduling Algorithm

Advantages:

- Implementation is easy.
- Requests do not have to wait in a queue.

Disadvantage:

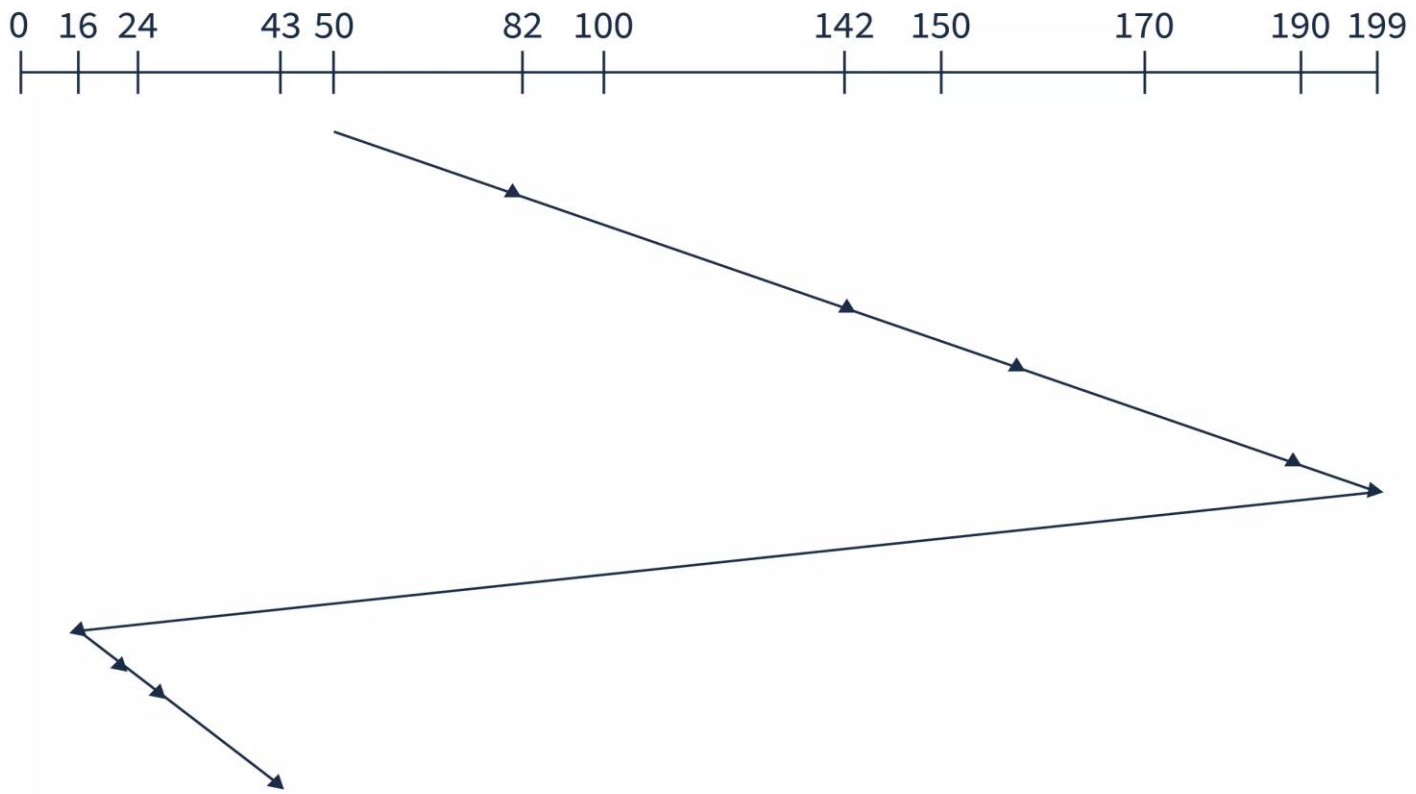
- The head keeps going on to the end even if there are no requests in that direction.

C-SCAN Disk Scheduling Algorithm

- It stands for "Circular-Scan".
- This algorithm is almost the same as the Scan disk algorithm but one thing that makes it different is that 'after reaching the one end and reversing the head direction, it starts to come back.
- The disk arm moves toward the end of the disk and serves the requests coming into its path.
- After reaching the end of the disk it reverses its direction and again starts to move to the other end of the disk but while going back it does not serve any requests.

C-SCAN Disk Scheduling Algorithm

Example: Suppose a disk having 200 tracks (0-199). The request sequence(82,170,43,140,24,16,190) of disk are shown as in the given figure and the head start is at request 50.



Seek time= (199-50) + (199-0) + (43-0) =391

C-SCAN Disk Scheduling Algorithm

Advantages:

- The waiting time is uniformly distributed among the requests.
- Response time is good in it.

Disadvantages:

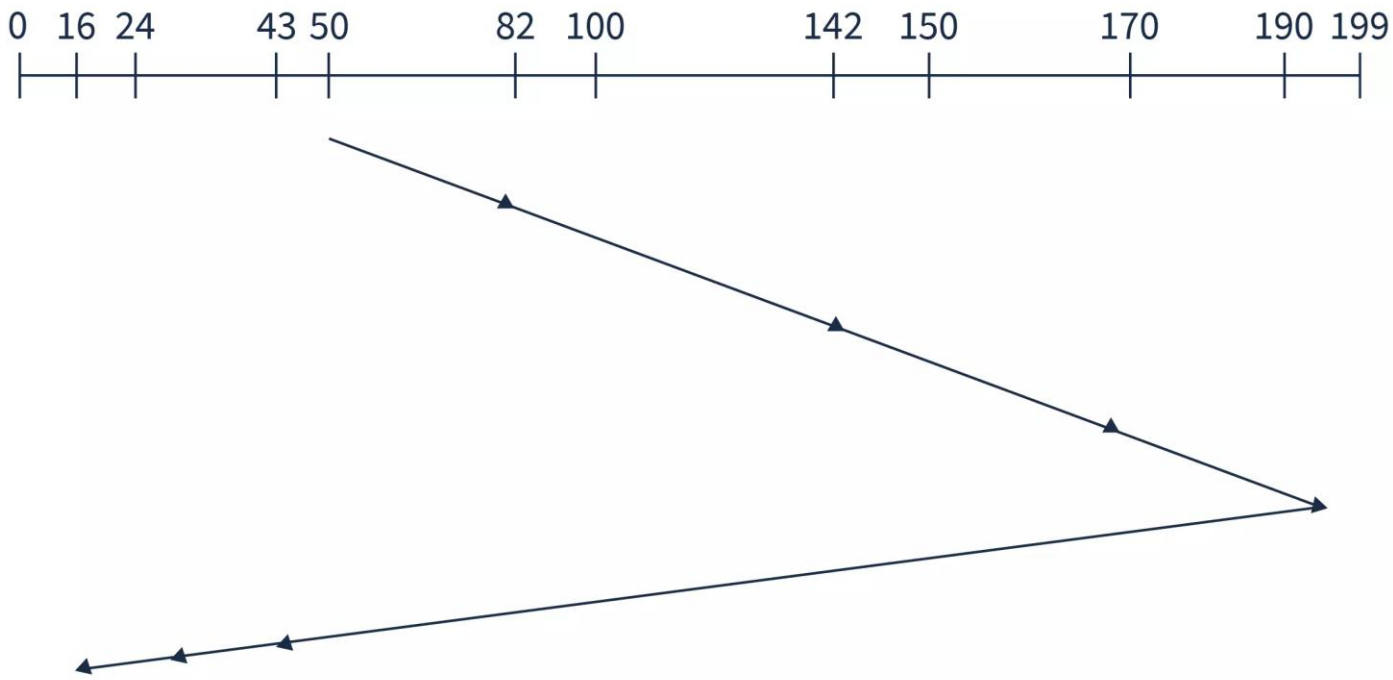
- The time taken by the disk arm to locate a spot is increased here.
- The head keeps going to the end of the disk.

LOOK Disk Scheduling Algorithm

In this algorithm, the disk arm moves to the 'last request' present and services them. After reaching the last requests, it reverses its direction and again comes back to the starting point. It does not go to the end of the disk, in spite, it goes to the end of requests.

LOOK Disk Scheduling Algorithm

Example: Suppose a disk having 200 tracks (0-199). The request sequence(82,170,43,140,24,16,190) of disk are shown as in the given figure and the head start is at request 50.



Seek time= (190-50) + (190-16) = 314

LOOK Disk Scheduling Algorithm

Advantages:

- Starvation does not occur.
- Since the head does not go to the end of the disk, the time is not wasted here.

Disadvantage:

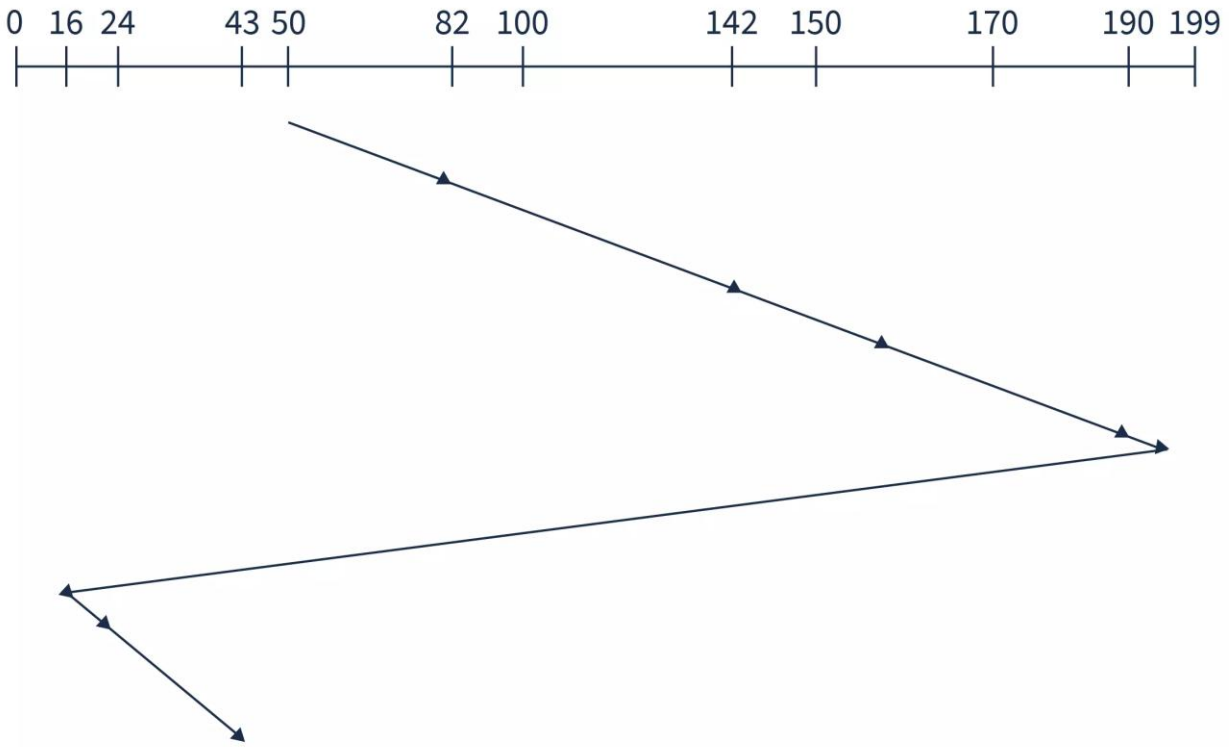
- The arm has to be conscious to find the last request

C-LOOK Disk Scheduling Algorithm

The C-Look algorithm is almost the same as the Look algorithm. The only difference is that after reaching the end requests, it reverses the direction of the head and starts moving to the initial position. But in moving back, it does not serve any requests.

C-LOOK Disk Scheduling Algorithm

Example: Suppose a disk having 200 tracks (0-199). The request sequence(82,170,43,140,24,16,190) of disk are shown as in the given figure and the head start is at request 50.



Seek time= (190-50) + (190-16) + (43-16) =341

C-LOOK Disk Scheduling Algorithm

Advantages:

- The waiting time is decreased.
- If there are no requests till the end, it reverses the head direction immediately.
- Starvation does not occur.
- The time taken by the disk arm to find the desired spot is less.

Disadvantage:

- The arm has to be conscious about finding the last request.